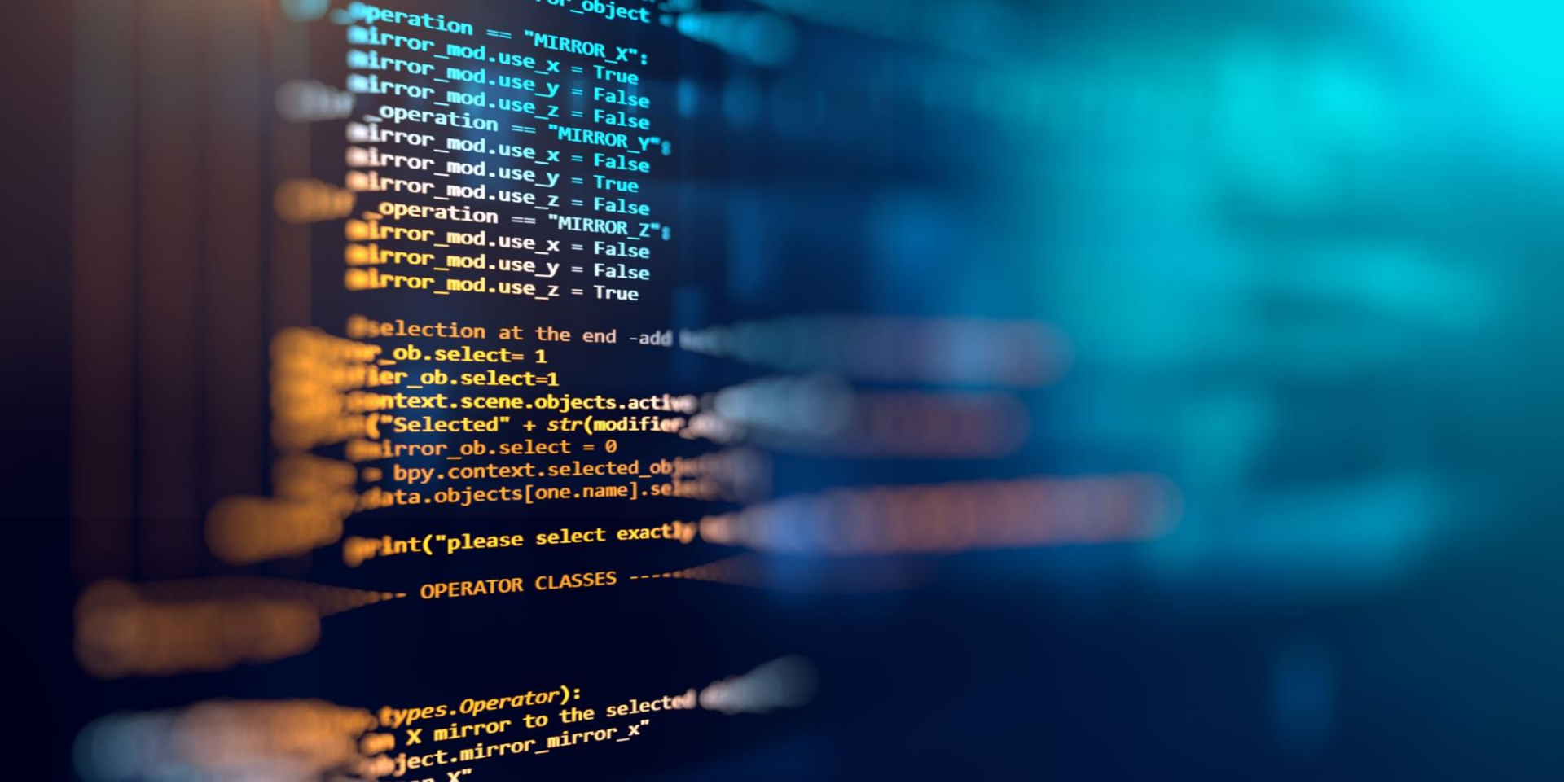


Q-LEARNING EXPLAINED

An Overview with Explanations

Q-LEARNING

Q-learning is a type of reinforcement learning algorithm used in machine learning. It helps an agent learn how to act optimally in a given environment by learning the value of actions in different states. The goal is to maximize the total reward over time.

Q-learning is a **model-free** reinforcement learning algorithm. This means it does not require a model of the environment to learn optimal actions. Instead, it learns directly from the interactions with the environment by updating a Q-value table that estimates the expected utility of taking a given action in a given state.

KEY CONCEPTS

Agent: The learner or decision maker.

Environment: The world through which the agent moves and interacts.

State (s): A specific situation in the environment.

Action (a): A choice made by the agent that affects the state.

Reward (r): Feedback from the environment based on the action taken.

Q-TABLE

Q-learning works by updating a **Q-table**, which is a table of values that represents the expected future rewards for each action in each state. The Q-table helps the agent decide the best action to take in any given state.

Q-TABLE

- **Rows**: Represent different states.
- **Columns**: Represent different actions.
- **Values**: Represent the expected future rewards for taking a particular action in a particular state.

The Q-value is updated using the bellman equation.

WHAT IS Q-VALUE?

Q-value (or action-value) represents the expected future reward of taking a specific action in a specific state, and then following the optimal policy thereafter. The Q-value helps the agent decide which action to take in each state to maximize its total reward over time.

Key Points:

- **State-Action Pair**: Each Q-value is associated with a specific state-action pair, denoted as Q(s,a).
- **Expected Reward**: The Q-value estimates the total expected reward starting from state *s*, taking action *a*, and then following the best policy.
- **Update Rule**: Q-values are updated iteratively using the Q-learning update rule: $Q(s,a) \leftarrow Q(s,a)+\alpha[r+\gamma \max Q(s',a')-Q(s,a)]$

Q-VALUE

The Q-value is updated using the formula:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]$$

Where:

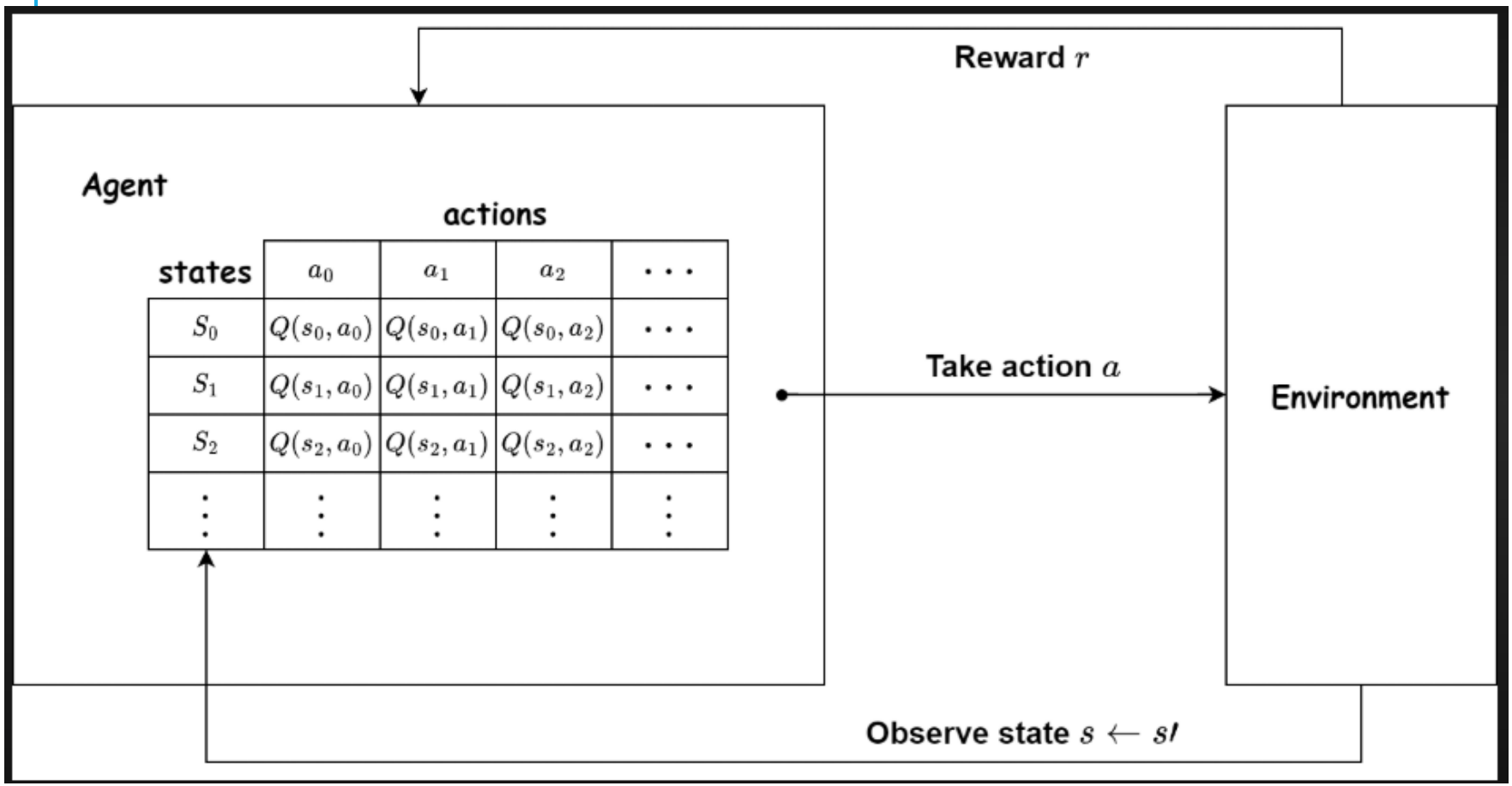
- α is the learning rate (how much new information overrides the old).
- γ is the discount factor (how much future rewards are valued compared to immediate rewards).
- r is the reward received after taking action a in state s.
- s' is the new state after taking action a.

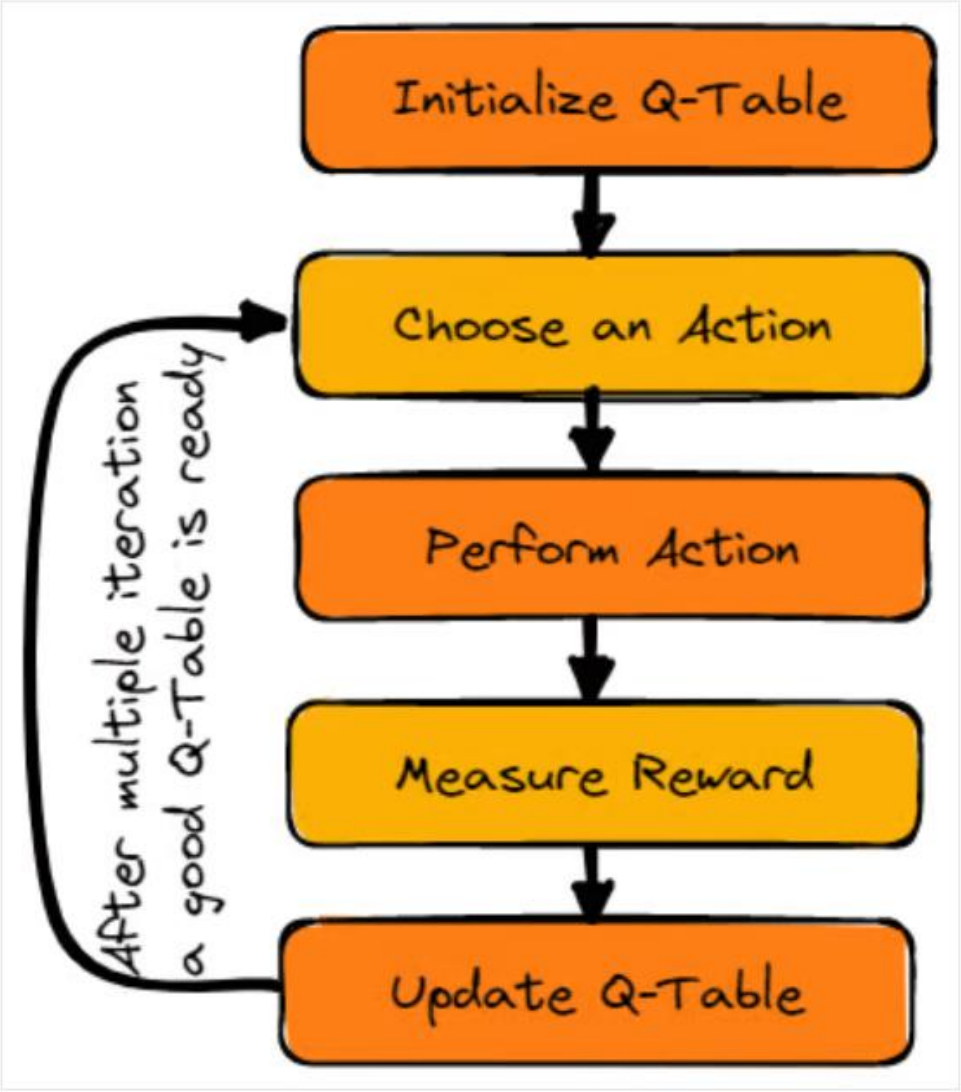
Q-FUNCTION

The Q-function uses the Bellman equation and takes state(s) and action(a) as input. The equation simplifies the state values and state-action value calculation.

BELLMAN FORD EQUATION

https://www.researchgate.net/figure/Bellman-Equation-of-Q-Learning_fig5_365205274


Reward r
Agent
actions
states
a_0
a_1
a_2
S_0
$Q(s_0, a_0)$
$Q(s_0, a_1)$
$Q(s_0, a_2)$
S_1
$Q(s_1, a_0)$
$Q(s_1, a_1)$
$Q(s_1, a_2)$
S_2
$Q(s_2, a_0)$
$Q(s_2, a_1)$
$Q(s_2, a_2)$
Take action a
Environment
Observe state $s \leftarrow s'$



https://www.datacamp.com/tutorial/introduction-q-learning-beginner-tutorial

STEPS

1. **Initialize Q-values**: Start by initializing the Q-values for all state-action pairs. This can be done arbitrarily, often set to zero.
2. **Choose an action**: Select an action using a policy derived from the Q-values. Commonly, an epsilon-greedy policy is used, where most of the time the action with the highest Q-value is chosen, but occasionally a random action is selected to ensure exploration.
3. **Take the action**: Execute the chosen action in the environment.
4. **Observe the outcome**: Record the reward received and the new state resulting from the action.
5. **Update Q-values**: Update the Q-value for the state-action pair using the bellman formula using
   1. Q(s,a) is the current Q-value,
   2. α is the learning rate,
   3. r is the reward received,
   4. γ is the discount factor,
   5. $\max_{a'} Q(s',a')$ is the maximum Q-value for the next state s'.
6. **Repeat**: Repeat steps 2-5 until the Q-values converge or for a specified number of iterations

EPSILON-GREEDY

The epsilon-greedy strategy is a popular method in reinforcement learning to balance exploration and exploitation.

EPSILON-GREEDY

How It Works

1. **Exploitation**: With probability $1-\epsilon$, the agent chooses the action that has the highest estimated Q-value for the current state. This means the agent is exploiting its current knowledge to maximize the reward.
2. **Exploration**: With probability ϵ, the agent chooses a random action. This means the agent is exploring the environment to discover potentially better actions that haven't been tried much.

Example

If $\epsilon=0.1$, the agent will:

- Choose the best-known action 90% of the time $(1-\epsilon)$.
- Choose a random action 10% of the time (ϵ).

LEARNING RATE

Learning Rate (α)

- **Range**: Typically between 0 and 1.
- **Purpose**: Controls how much new information overrides old information. A higher learning rate means the agent learns faster but may be less stable, while a lower learning rate means the agent learns more slowly but may be more stable.
- **Common Values**: Can be set between 0.001 and 0.5. For example, $\alpha=0.1$ is a common starting point.

DISCOUNT FACTOR

Discount Factor (γ)

- **Range**: Typically between 0 and 1.
- **Purpose**: Determines the importance of future rewards. A higher discount factor means the agent values future rewards more, while a lower discount factor means the agent focuses more on immediate rewards.
- **Common Values**: Often set between 0.9 and 0.99. For example, $\gamma=0.9$ is a common choice for balancing immediate and future rewards.

SIMULATOR

https://www.mladdict.com/q-learning-simulator

WHEN DOES Q-LEARNING STOP?

1. Convergence:

- **Q-values Stabilize**: The algorithm stops when the Q-values for all state-action pairs converge and no longer change significantly with further updates.

2. Fixed Number of Iterations:

- **Predefined Iterations**: The algorithm can be set to run for a fixed number of iterations or episodes. Once this limit is reached, the learning process stops.

3. Performance Criteria:

- **Goal Achievement**: The algorithm stops when the agent consistently achieves the desired performance level, such as reaching the goal state or achieving a certain reward threshold.

4. Time Limit:

- **Maximum Time**: The learning process can be limited by a maximum time duration. Once this time limit is reached, the algorithm stops.

APPLICATIONS

Q-learning is used in a variety of applications across different fields. Here are some notable examples:

Robotics:

- **Navigation**: Robots use Q-learning to navigate through environments, avoiding obstacles and reaching target locations efficiently.
- **Control**: Q-learning helps in controlling robotic arms for tasks like assembly and manipulation.

Gaming:

- **Game AI**: Q-learning is used to develop intelligent agents that can play games, learn strategies, and adapt to different game scenarios.

Network Optimization:

- **Traffic Control**: Q-learning is used to optimize network traffic signals, reducing congestion and improving flow1
- **Resource Allocation**: It helps in efficiently allocating resources in communication networks.

Autonomous Vehicles:

- **Path Planning**: Autonomous vehicles use Q-learning to plan optimal paths, avoid obstacles, and navigate complex environments.

Energy Management:

- **Smart Grids**: Q-learning helps in managing energy distribution in smart grids, optimizing the use of renewable energy sources.

MUST READ

https://www.datacamp.com/tutorial/introduction-q-learning-beginner-tutorial

RESOURCES

https://sefidian.com/2021/03/01/policy-g/

https://pylessons.com/A2C-reinforcement-learning

https://dilithjay.com/blog/actor-critic-methods

https://lilianweng.github.io/posts/2018-04-08-policy-gradient/

https://web.stanford.edu/~ashlearn/RLForFinanceBook/PolicyGradient.pdf

https://www.tutorialspoint.com/machine_learning/machine_learning_temporal_difference_learning.htm

https://en.wikipedia.org/wiki/Temporal_difference_learning

https://www.datacamp.com/tutorial/introduction-q-learning-beginner-tutorial